

If I say, I want to start development from start. give me step by step instructions



Here is a build order that avoids the problems you hit earlier. The rule throughout: finish one small step, test it in the browser, commit it, then move on.

## Phase 0: Setup (30 minutes)

1. **Install** VS Code, the **Live Server** extension, and Git.
2. **Create the folder structure:**

```
aams/
├── index.html
├── css/
│   └── design.css
├── js/
│   └── script.js
└── photos/
    └── cou_logo.png
```

Name the main file `index.html` , not `home.html` , because hosting services look for that name.

3. **Create a GitHub repo** and run `git init` , then commit and push. Commit after every working step.
4. **Keep the browser console open (F12)** the whole time. It would have caught your first JS error immediately.

## Phase 1: Empty skeleton

Goal: the navigation works before any real content exists.

1. In `index.html` , write a clean document with the `<!DOCTYPE html>` on line 1, a `<head>` that links the CSS, and the script at the end of the body: `<script src="js/script.js" defer></script>` .
2. Add the navbar and **empty sections**, each with only a heading and the classes `page-section` `hidden` : `home` , `dashboard` , `notices` , `guideline` , `application-form` , `helpline` , `login` .
3. In `design.css` , add only the reset, the navbar, `.hidden` and `.page-section` .
4. In `script.js` , write only `navigateTo()` and the click handlers for the nav buttons.

**Done when:** clicking each nav button shows its section and hides the others, with no errors in the console.

## Phase 2: Static pages, one at a time

Build them in this order and test each one: Home (hero and buttons), then Important Dates, Notices, Guideline, Helpline/FAQ, and the footer.

**Done when:** the whole site looks right on desktop and on a phone. Use F12 and then the device toolbar to check the phone view.

## Phase 3: Login and registration (frontend only)

1. Build the login and registration forms with HTML and CSS.
2. In JS, add the `isLoggedIn` state and `updateNav()` .
3. Use `localStorage` as a fake database for now, so you can test the flow without a backend: register saves a user, login checks it.
4. Show or hide the Login, Dashboard and Logout buttons based on the state.

**Done when:** you can register, log in, see the dashboard, log out, and the nav buttons change correctly.

## Phase 4: Application form and dashboard

1. Extend the form: personal information, SSC and HSC details, unit selection, photo upload.
2. Add **validation** in JS: required fields, GPA ranges, eligibility per unit.
3. On submit, save the application and show its real status on the dashboard instead of the hardcoded "Under Review".
4. Add the admit card page: a printable layout using `@media print` .

**Done when:** a logged-in user can apply once, see the status, and print an admit card.

## Phase 5: Backend and database

Only now replace `localStorage` . Pick one:

| Option                                      | Good for                                              |
|---------------------------------------------|-------------------------------------------------------|
| <b>Firebase</b> (Auth + Firestore)          | Fastest, little backend code                          |
| <b>Node.js + Express + MongoDB or MySQL</b> | Best for learning and for showing in a project report |

Replace each `localStorage` call with an API call, one feature at a time: register, login, application, dashboard.

## Phase 6: Admin features

An admin login, then the ability to publish notices, view applications, enter test results and generate merit lists. This is what makes it an "automated" system.

## Phase 7: Test and deploy

1. Test the whole flow with several fake users.
2. Fix mobile problems.
3. Deploy the frontend on GitHub Pages, Netlify or Firebase Hosting, and the backend on Render or similar.

## Habits that will save you time

- Always keep **one version of the files**. Don't keep `home.html` and `index.html` side by side.
- **Name things consistently**. Your JS errors came from `navNotice` versus `navNotices` . If you keep the ID, the variable and the function name the same, such as `nav-notice` , `navNotice` and `navigateTo("notices")` , that mistake goes away.
- Don't build the next phase until the current "Done when" passes.

I'd start with Phase 0 and Phase 1 today. Do you want me to write the clean `index.html` , CSS and JS for Phase 1, so you have a correct base to start from?